

SENAI 2026

Relatório de Banco de Dados

Vanessa Samanta Barp

08/10/2026

INTRODUÇÃO

A realização do banco de dados foi desenvolvida durante a aula com o auxílio do Laragon. O Laragon permite trabalhar principalmente com dados estruturados, que são organizados e armazenados em bancos de dados relacionais, como o MySQL. A utilização do Laragon facilita a configuração e o gerenciamento das ferramentas necessárias para o desenvolvimento de sistemas, como PHP, MySQL e Apache. Dessa forma, é possível criar, armazenar, consultar e testar informações de maneira mais rápida e organizada. Além disso, o Laragon é uma ferramenta leve, gratuita e de fácil utilização, tornando-se uma opção bastante prática para nós estudantes que estamos aprendendo sobre programação e bancos de dados. Durante a aula, seu uso contribuiu para colocar em prática os conhecimentos adquiridos sobre a criação e organização de um banco de dados.

1.0_AS PROCEDURES

A procedure `sp_atualizar_produto` tem como função atualizar o nome de um produto já cadastrado na tabela `produtos`. Ela recebe como parâmetros o identificador do produto e o novo nome que será utilizado. Por meio do comando `UPDATE`, a procedure localiza o produto pelo seu ID e realiza a alteração do nome. Dessa forma, ela facilita a atualização dos dados e evita a necessidade de repetir manualmente o comando SQL sempre que for necessário modificar um produto.

Este é o código utilizado para a stored procedures `sp_atualizar_produto`:

```
BEGIN
    UPDATE produtos
    SET nome = p_novo_nome
    WHERE id = p_id_produto;
END
```

A procedure `sp_registrar_compra` tem como função registrar uma nova compra na tabela `compras`. Ela recebe como parâmetro o identificador do fornecedor, o identificador do produto, a quantidade e o código da compra. Por meio do comando `INSERT INTO`, a procedure adiciona essas informações na tabela, registrando também automaticamente a data da compra. Dessa forma, ela facilita o cadastro de novas compras e evita a necessidade de repetir manualmente o comando SQL sempre que uma compra for registrada.

Este é o código utilizado para a stored procedures `sp_registrar_compra` :

```
BEGIN
    INSERT INTO compras (id_fornecedor,
                          id_produto,
                          quantidade,
                          codigo,
                          data_compra)
    VALUES (
        p_id_fonecedor,
        p_i_produto,
        p_quantidade,
        p_codigo,
        NOW()
    );
END
```

A procedure `sp_compras_por_fornecedor` tem como função consultar as compras realizadas por um determinado fornecedor. Ela recebe como parâmetros o identificador do fornecedor e uma data inicial e final. Por meio do comando `SELECT`, a procedure busca o código da compra, o nome do produto, a quantidade, a data da compra e o nome do fornecedor. Dessa forma, facilita a consulta das compras de um fornecedor dentro de um período específico.

Este é o código utilizado para a stored procedures `sp_compras_por_fornecedor`:

```
BEGIN
    SELECT
        c.codigo,
        p.nome AS produto,
        c.quantidade,
        c.data_compra,
        f.nome AS fornecedor
    FROM compras c
    INNER JOIN fornecedores f ON f.id = c.id_fornecedor
    INNER JOIN produtos p ON p.id = c.id_produto
    WHERE c.id_fornecedor = p_id_fornecedor
    AND c.data_compra BETWEEN p_data_inicio AND p_data_fim;
END
```

2.0_AS TABELAS

A tabela clientes é responsável por armazenar as principais informações dos clientes. Na imagem, são apresentados 5 exemplos de registros do meu banco de dados. A tabela possui os campos ID, que identifica cada cliente de forma única; nome, que apresenta o nome completo; CPF/CNPJ, que armazena o documento do cliente; rua, número e bairro, que correspondem às informações do endereço; telefone, utilizado para contato; CEP, que identifica a localização; e referência, que apresenta um ponto de referência para facilitar a localização do endereço. Dessa forma, a tabela permite organizar e consultar de maneira simples as informações dos clientes cadastrados.

Este é um exemplo de como ficaria com os dados já inseridos :

id	nome	cpf_cnpj	rua	telefone	numero	bairro	cep	referencia
1	Ana Beatriz Souza	12345678901	Rua das Flores	47991234567	120	Centro	89200001	Próximo à praça central
2	Bruno Henrique Lima	23456789012	Rua São Paulo	47992345678	245	Vila Nova	89200002	Ao lado da escola
3	Camila Oliveira Santos	34567890123	Rua das Palmeiras	47993456789	87	São Pedro	89200003	Perto do mercado
4	Daniel Ferreira Costa	45678901234	Rua XV de Novembro	47994567890	310	Centro	89200004	Em frente à igreja
5	Eduarda Martins Silva	56789012345	Rua Amazonas	47995678901	56	América	89200005	Próximo ao posto

A tabela compras é responsável por armazenar os registros das compras realizadas no banco de dados. Na imagem, são apresentados 5 exemplos de registros. O campo ID identifica cada compra, o id_fornecedor indica qual fornecedor realizou o fornecimento, o id_produto identifica o produto comprado, a quantidade informa o número de unidades adquiridas, o código identifica a compra e a data_compra registra a data em que a compra foi realizada. Dessa forma, a tabela permite organizar e acompanhar as compras realizadas, relacionando os fornecedores aos produtos adquiridos.

Este é um exemplo de como ficaria com os dados já inseridos :

id	id_fornecedor	id_produto	quantidade	codigo	data_compra
1	1	10	5	5001	2026-01-05 00:00:00
2	2	3	12	5002	2026-01-06 00:00:00
3	3	15	8	5003	2026-01-07 00:00:00
4	4	7	20	5004	2026-01-08 00:00:00
5	5	1	2	5005	2026-01-09 00:00:00

A tabela fornecedores é utilizada para armazenar as informações dos fornecedores cadastrados no banco de dados. Na imagem, são apresentados 5 exemplos de fornecedores. O campo ID identifica cada fornecedor, enquanto o nome apresenta seu nome ou razão social e o CPF/CNPJ armazena seu documento. Os campos rua, número, bairro e CEP registram as informações do endereço, enquanto o campo telefone armazena o contato do fornecedor. Dessa forma, a tabela mantém os dados dos fornecedores organizados e facilita sua identificação e consulta no sistema.

Este é um exemplo de como ficaria com os dados já inseridos :

id	nome	cpf_cnpj	rua	numero	bairro	cep	telefone
1	Distribuidora Vale Sul	12345678000101	Rua das Indústrias	120	Centro	89201001	47991234001
2	Comercial Santa Catarina	23456789000112	Rua Joinville	245	América	89201002	47992345002
3	Atacadista Bom Preço	34567890000123	Rua Blumenau	87	Vila Nova	89201003	47993456003
4	Alimentos Brasil Ltda	45678901000134	Rua Paraná	310	Boa Vista	89201004	47994567004
5	Distribuidora São José	56789012000145	Rua São Paulo	156	Floresta	89201005	47995678005

A tabela de pedidos é utilizada para armazenar as informações sobre os pedidos e itens registrados no banco de dados. Na imagem, são apresentados 5 exemplos de registros. O campo id é a chave primária que identifica unicamente cada registro da tabela. Os campos id_clientes e id_produtos são chaves estrangeiras que fazem o vínculo com o cliente que fez a compra e o produto adquirido, respectivamente. O campo num_ped indica o número identificador do pedido, enquanto quantidade registra o total de unidades compradas e data_ped armazena a data e o horário em que o pedido foi realizado. Dessa forma, a tabela mantém o histórico de vendas organizado e facilita o controle e a consulta das compras no sistema.

Este é um exemplo de como ficaria com os dados já inseridos :

id	id_clientes	id_produtos	num_ped	quantidade	data_ped
1	1	10	1001	2	2026-01-05 00:00:00
2	2	5	1002	1	2026-01-06 00:00:00
3	3	12	1003	5	2026-01-07 00:00:00
4	4	8	1004	3	2026-01-08 00:00:00
5	5	1	1005	10	2026-01-09 00:00:00

A tabela de produtos é utilizada para armazenar as informações sobre os itens cadastrados no banco de dados. Na imagem, são apresentados 5 exemplos de produtos. O campo id é a chave primária que identifica unicamente cada produto na tabela. O campo cod_barras armazena o código de barras numérico do item, enquanto nome indica a denominação do produto e descrição detalha suas especificações gerais. O campo estoque indica a quantidade de unidades disponíveis em inventário, o campo valor armazena o preço de venda, e os campos data_fabricacao e data_validade registram, respectivamente, a data de fabricação e a data de expiração do item. Dessa forma, a tabela mantém o controle do inventário organizado e facilita a consulta e a gestão dos produtos no sistema.

Este é um exemplo de como ficaria com os dados já inseridos :

id		cod_barras	nome	descricao	estoque	valor	data_fabricacao	data_validade
1		7891000000011	Arroz barborizado 5kg	Arroz branco tipo 1 pacote 5kg	76	28,9	2026-01-10 00:00:00	2027-01-10 00:00:00
2		7891000000028	Feijão Carioca 1kg	Feijão carioca tipo 1 pacote 1kg	65	8,9	2026-02-15 00:00:00	2027-02-15 00:00:00
3		7891000000035	Açúcar Cristal 2kg	Açúcar cristal refinado pacote 2kg	90	7,5	2026-03-05 00:00:00	2027-03-05 00:00:00
4		7891000000042	Café Torrado 500g	Café torrado e moído pacote 500g	55	16,9	2026-01-20 00:00:00	2027-01-20 00:00:00
5		7891000000059	Farinha de Trigo 1kg	Farinha de trigo tradicional pacote 1kg	70	5,99	2026-02-10 00:00:00	2027-02-10 00:00:00

2.0_AS VIEWS

A view view_clientes_com_mais_compras serve para mostrar os clientes que mais fizeram compras no sistema. Ela conta quantas compras cada cliente realizou e organiza os resultados do maior para o menor. O campo nome mostra o cliente e total_compras mostra a quantidade de compras realizadas.

Segue exemplo de como ficaria os dados da view:

nome	total_compras
Bruno Henrique Lima	4
Camila Oliveira Santos	4
Eduarda Martins Silva	4
Henrique Gomes Alves	4
João Pedro Martins	4

A view `producao.view_compra_detalhada` é utilizada para consultar as compras efetuadas aos fornecedores de forma simplificada. O campo `id_compra` identifica o registo, `codigo` indica o número da transação e `data_compra` armazena a data da compra. O campo `produto` mostra o item adquirido, enquanto `fornecedor` e `documento_fornecedor` exibem o nome do fornecedor e o seu CNPJ. Por fim, `quantidade_compra` regista o total de unidades compradas.

Segue exemplo de como ficaria os dados da view:

id_compra	codigo	data_compra	produto	fornecedor	documento_fornecedor	quantidade_compra
1	5001	2026-01-05 00:00:00	Biscoito Chocolate	Distribuidora Vale Sul	12345678000101	5
2	5002	2026-01-06 00:00:00	Açúcar Cristal 2kg	Comercial Santa Catarina	23456789000112	12
3	5003	2026-01-07 00:00:00	Queijo Mussarela 500g	Atacadista Bom Preço	34567890000123	8
4	5004	2026-01-08 00:00:00	Óleo de Soja 900ml	Alimentos Brasil Ltda	45678901000134	20
5	5005	2026-01-09 00:00:00	Arroz barborizado 5kg	Distribuidora São José	56789012000145	2

A view `producao.view_consolidação_de_compras_por_fornecedor` é utilizada para resumir o histórico de compras agrupado por fornecedor. O campo `fornecedor` exibe o nome da empresa fornecedora, `total_compras` mostra a quantidade total de pedidos feitos a ela, `quantidade_total_fornecida` soma o total de itens adquiridos e `ultima_compra` regista a data do pedido mais recente.

Segue exemplo de como ficaria os dados da view:

fornecedor	total_compras	quantidade_total_fornecida	ultima_compra
Comercial Horizonte	7	60	2026-09-10 00:00:00
Comercial Santa Catarina	4	51	2026-04-02 00:00:00
Atacadista Bom Preço	4	52	2026-04-12 00:00:00
Distribuidora São José	4	60	2026-04-09 00:00:00
Atacado União	4	48	2026-04-06 00:00:00

A view `producao.view_de_demanda_de_produtos` é utilizada para analisar a procura por cada produto no sistema. O campo `produto` exibe o nome do item, `total_de_entregas` mostra o número de entregas realizadas e `total_itens_comprados` indica a quantidade total de unidades vendidas/compradas deste produto.

Segue exemplo de como ficaria os dados da view:

produto	total_de_entregas	total_itens_comprados
Feijão Carioca 1kg	5	116
Leite em Pó 400g	5	97
Água Mineral 1,5L	5	95
Farinha de Trigo 1kg	5	90
Queijo Mussarela 500g	5	82

view_fornecedores_com_mais_vendas: Esta consulta cruza as tabelas de vendas e fornecedores para identificar quais parceiros comerciais ou fornecedores geraram o maior volume de negócios ou maior faturamento acumulado. É útil para relatórios de desempenho de parceiros, negociações de contrato e identificação dos fornecedores mais estratégicos para a empresa.

Segue exemplo de como ficaria os dados da view:

fornecedor	total_vendas
Comercial Horizonte	7
Comercial Santa Catarina	4
Atacadista Bom Preço	4
Distribuidora São José	4
Atacado União	4

view_pedidos_com_valor_maior_que_media: Essa *view* realiza um cálculo dinâmico da média do valor de todos os pedidos registados no banco de dados e, em seguida, filtra apenas as transações individuais cujo montante total ultrapassa essa média geral. Serve para analisar transações de ticket alto, identificar perfis de clientes de maior valor e monitorizar vendas fora da curva padrão.

Segue exemplo de como ficaria os dados da view:

produto	total_de_entregas	total_itens_comprados
Feijão Carioca 1kg	5	116
Leite em Pó 400g	5	97
Água Mineral 1,5L	5	95
Farinha de Trigo 1kg	5	90
Queijo Mussarela 500g	5	82

view_produtos_mais_vendidos_no_ultimo_mes: Esta consulta filtra as saídas do inventário ou os itens dos pedidos realizados especificamente no intervalo dos últimos 30 dias (ou mês corrente), agrupando-os por produto e ordenando-os da maior para a menor quantidade vendida. Auxilia no controle de estoque, planejamento de reposição e análise de tendências de consumo recentes.

Segue exemplo de como ficaria os dados da view:

produto	total_vendas	data_ped
Arroz Branco 5kg	2	09/2026

2.0_AS TRIGGER

A Trigger `trg_atualiza_estoque_compras` vai atualizar o estoque toda vez que alguma compra for realizada.

segue código utilizado para criação da Trigger:

```
BEGIN
    UPDATE produtos
    SET estoque = estoque + NEW.quantidade
    WHERE id = NEW.id_produto;
END
```

A Trigger `trg_atualiza_estoque_pedidos` vai atualizar o estoque toda vez que algum pedido for realizado.

segue código utilizado para a criação da trigger:

```
BEGIN
    UPDATE produtos
    SET estoque = estoque - NEW.quantidade
    WHERE id = NEW.id_produtos;
END
```

A Trigger trg_produtos_atualiza vai atualizar o nome do produto após alguma mudança ser realizada.

segue código utilizado para a criação da trigger:

BEGIN

```
INSERT INTO tmp_historico_produtos(  
    produto_id,  
    nome_antigo,  
    preco_antigo,  
    estoque_antigo,  
    usuario_alteracao  
) VALUES (  
    OLD.id,  
    OLD.nome,  
    OLD.valor,  
    OLD.estoque,  
    USER()  
);
```

END